

VISITA GUIDATA · KIT ESSENZIALE

Metnos

*Un'architettura per agenti governati,
estensibili e verificabili.*



VISITA GUIDATA E KIT ESSENZIALE

Metnos

*Un'architettura ospitata in proprio per un **agente governato ed estensibile**. L'istanza di riferimento vive su un server Linux domestico, pianifica con un LLM locale, **può costruire candidati per gli strumenti che le mancano** dentro un vocabolario chiuso e governato, **chiede una decisione** quando autorità e rischio lo richiedono e distingue sempre un esito osservato da un tentativo o da un risultato parziale.*

mētis (μῆτις) «intelligenza astuta» + noûs (νοῦς) «mente»

I/O	chiuso	traccia	IT + EN	0
CONTRATTI EXECUTOR	VOCABOLARIO	AZIONI ED ESITI	TESTI E ISTRUZIONI	CLOUD RICHIESTO PER
STABILI	GOVERNATO	RISCONTRABILI	LOCALIZZATI	PENSARE

stato: pre-1.0, installabile e operativo · **installazione:** programma incluso nel repository · **licenza:** AGPL-3.0 · **codice:** github.com/brunialti/metnos

IN QUESTA PAGINA

- 01 Cos'è Metnos, in trenta secondi
- 02 Perché è diverso
- 03 Nove casi, un solo sistema
- 04 Il Tutor: Metnos sa spiegare Metnos
- 05 Dove vive e come gli si parla
- 06 Cosa succede quando dice «posso?»
- 07 La sua memoria
- 08 Come cresce: Synt, skill e servizi
- 09 Persone e dispositivi
- 10 Cosa abilita il catalogo, cosa vieta il core
- 11 Stato operativo
- 12 Glossario minimo & letture

1 · Cos'è Metnos, in trenta secondi

Gli si affidano obiettivi in linguaggio naturale dal **browser**, da **Telegram** o da un altro canale ammesso. L'installazione di riferimento legge mail, file, foto, calendario, web e GitHub, ma sono esempi, non il confine: ogni capacità governabile che possa essere espressa come executor firmato può entrare nella stessa architettura. Prima delle azioni che richiedono consenso applica regole e conferme esplicite. Non c'è un cloud di Metnos e non c'è un «registrati»: dati e controllo restano all'operatore.

Quello che succede a ogni richiesta, in un disegno:

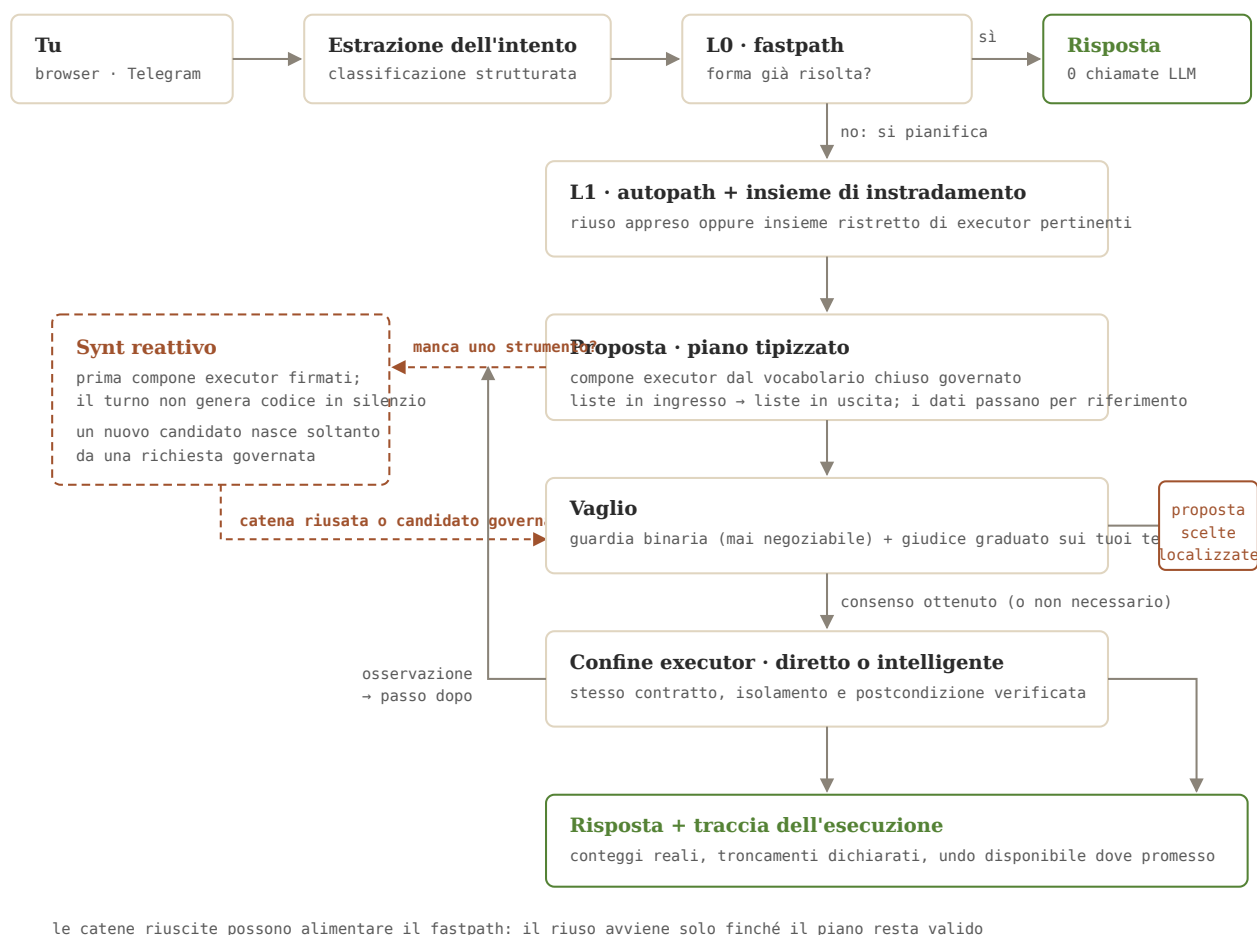


Fig. 1 — Il giro di un turno. L0 e L1 possono riusare piani ancora validi; negli altri casi il proponente lavora su un insieme ristretto. L'intelligenza interna di un executor non cambia il contratto visto dal pianificatore.

Sei proprietà che lo distinguono da un agente generico:

- **Chiede prima di agire.** Un valutatore in due fasi — guardia binaria, poi giudice graduato sui vostri telos — decide se un'azione passa in silenzio, chiede conferma o viene rifiutata. Quando serve il consenso, il canale presenta una proposta localizzata con le scelte disponibili. Vedi vaglio.
- **Può costruire gli strumenti che gli mancano.** Niente catalogo infinito scritto a mano: durante il turno Synt prova prima a comporre executor già firmati; la generazione di un candidato nuovo parte soltanto da una richiesta esplicita e attraversa nome governato, prove, firma e ciclo di ammissione. Vedi Synt e la scena 6.

- **Può agentificare un executor senza agentificare il piano.** Se il percorso è incerto ma lo scopo è stretto, un LLM interno disambigua i passi entro azioni e limiti fissati. Per il pianificatore resta lo stesso executor tipizzato. Executor intelligenti.
- **Porta l'esecuzione dove vivono i dati.** Un executor compatibile può essere eseguito su un PC registrato; decisione, consenso, firma e tracciamento restano sul server. Il dispositivo esegue, non diventa un secondo agente. Executor remoti.
- **Separa i controlli deterministici dalla generazione probabilistica.** Vocabolario, firme, guardie, regole, identità delle memorie dei piani e verifiche di validità sono testabili. La generazione di un piano nuovo e la prosa finale restano invece risultati di un modello e non vengono dichiarate riproducibili.
- **Non mente sull'esito.** I conteggi riflettono ciò che è stato *davvero* fatto; i troncamenti vengono dichiarati; ogni azione che si dichiara annullabile lo è per costruzione (catalogo chiuso di schemi di inversione).

2 · Perché è diverso

Di agenti ce ne sono già, a famiglie intere — le infrastrutture da assemblare, gli agenti da terminale, i maggiordomi in chat (*OpenClaw* e simili) e l'ecosistema delle «skill» importabili. Metnos fa scelte diverse su più assi, e le fa per una ragione precisa: **un pianificatore locale non è un modello di frontiera**, e va messo nelle condizioni di non sbagliare.

	INFRASTRUTTURA AGENTICA TIPICA	METNOS
Strumenti	Scritti a mano, importati o generati al volo in forma libera — poi eseguiti così come sono, coi privilegi dell'assistente.	Possono essere sintetizzati dal sistema, ma dentro un vocabolario chiuso e verificato : firmati, provati alla nascita, sottoposti a maturazione e ammessi solo dopo un filtro a sette livelli.
Instradamento	Il modello sceglie uno strumento a ogni turno: chiedete due volte, ottenete due piani.	Governato e riusabile : vocabolario chiuso, cache di piani con identità e validità esplicite, controlli deterministici. Un piano nuovo prodotto da un LLM può variare.
Risultato	Forma libera, per strumento.	Uniforme: liste in ingresso , liste in uscita , componibili fra passi senza colla.
Intelligenza interna	Concentrata nell'agente generale, che sceglie obiettivi e strumenti.	Può vivere anche dentro un executor a mandato ristretto, senza cambiame ingresso, uscita o autorità.
Collocazione	Gli strumenti girano dove vive l'agente.	Un executor compatibile può girare su un dispositivo registrato; il server conserva decisione e tracciamento.
Annullamento	Raro, o nella migliore delle ipotesi «si spera».	Funzione fondamentale: catalogo chiuso di operazioni inverse, spostamento = COPIA → verifica → CANCELLAZIONE, <code>ok_count</code> aderente alle operazioni realmente annullate.
Lingua	Inglese, stringhe cablate nel codice.	Localizzazione per costruzione: ogni stringa e istruzione al modello è un <i>dato distinto per lingua</i> . Italiano e inglese sono convalidati; le altre lingue si aggiungono come pacchetti di traduzione e non sono ancora collaudate.
Sicurezza	Ci si fida dell'autore del pacchetto.	<i>Non</i> ci si fida del pacchetto: è il pacchetto a dover passare i controlli.

Il contesto architetturale

I sistemi agentici possono lasciare al modello la scelta libera degli strumenti, oppure racchiuderla in grafi, permessi e ambienti isolati. Metnos sceglie un confine più stretto: il pianificatore vede soltanto capacità tipizzate e ammesse; l'origine dello strumento, locale, importata o remota, non ne amplia l'autorità.

Questa scelta sacrifica parte della velocità con cui si possono collegare strumenti arbitrari. In cambio rende ispezionabili provenienza, regole, consenso, collocazione dell'esecuzione e tracciamento. Non trasforma però un LLM in una funzione pura: i controlli e il riuso dei piani sono verificabili; la generazione nuova e la formulazione della risposta possono variare.

Due scelte che cambiano il perimetro

Non sono due nuovi strumenti: sono due modi generali di far lavorare gli executor senza allargare il linguaggio del pianificatore.

INCERTEZZA DENTRO UN CONFINO

Agente dentro, executor fuori

Un executor **agentificato** riceve lo stesso ingresso tipizzato e deve produrre la stessa uscita di un executor diretto. La differenza è interna: può osservare, scegliere un passo e riosservare quando il percorso non è noto in anticipo.

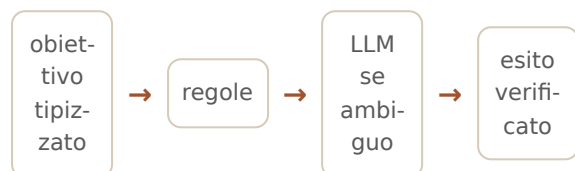
Agente generale

Scompone obiettivi, sceglie strumenti e può attraversare domini.

Executor agentificato

Ha un solo incarico, azioni enumerate, budget e arresti fissati.

La distinzione è deliberata: chiamarlo semplicemente «agente» nasconderebbe chi può cambiare obiettivo o autorità. Metnos mette la capacità adattiva *dentro* un confine che il pianificatore può ancora comporre e verificare.



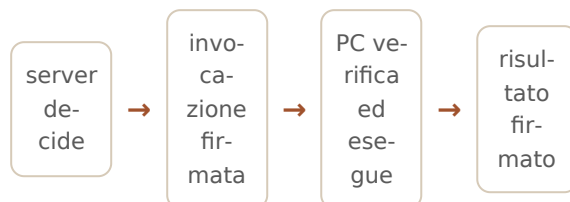
- Il LLM è un ripiego per il percorso, non l'autorità che decide lo scopo.
- Segreti, consenso e permessi restano fuori dal modello.
- Senza una postcondizione osservabile non può dichiarare successo.

Esempio: `login_sites` cerca l'area di accesso, gestisce form a uno o più passi e controlla l'esito; non può navigare verso un altro obiettivo. Come funziona.

CALCOLO VICINO AI DATI

Stesso executor, un'altra macchina

Un **executor remoto** non è una copia di Metnos e non è un secondo agente. Il server continua a pianificare, applicare regole e consenso, firmare il lavoro e registrare l'esecuzione; un client sottile esegue sul dispositivo registrato.



- I dati possono restare sul PC che li possiede.
- Identità, piattaforme ammesse e collocazione fanno parte del contratto.
- PC spento o revocato significa errore esplicito, mai esecuzione nascosta sul server.

Implicazione: la casa diventa una superficie di esecuzione distribuita, ma conserva un solo punto di decisione e responsabilità. Dettagli e limiti.

Due assi indipendenti: diretto o agentificato descrive *come* l'executor risolve il compito; locale o remoto descrive *dove* gira. Ogni executor deve comunque dichiarare esplicitamente quali combinazioni supporta.

diretto · locale

agentificato · locale

diretto · remoto

agentificato · remoto

Perché Metnos non ha adottato il formato «skill» standard

I formati di skill in voga — le skill in markdown, i server MCP — sono comodi e pericolosi in pari misura: importate un pacchetto e l'assistente *ne esegue il codice coi propri privilegi*. Per un assistente che tocca file, mail e shell di casa vostra, è esecuzione remota di codice per disegno: basta un pacchetto malevolo o sciatto. Metnos sceglie la **sicurezza per costruzione**: vocabolario chiuso (non potete nominare uno strumento che la grammatica non ammette), cancello di ammissione a 7 strati per ogni executor nuovo o importato (firma → sovrapposizione di affinità → maturazione → isolamento → prova operativa → verifica LLM → registro a sola aggiunta), consenso esplicito prima di ogni azione distruttiva. Il principio è semplice: *non fidarti del pacchetto; deve guadagnarsi il posto*. L'ecosistema pubblico delle skill non viene ignorato: viene tradotto dentro questo modello, mai eseguito crudo.

Il determinismo ha un perimetro preciso

Metnos rende deterministici i passaggi che devono essere controllabili: vocabolario, firme, regole di sicurezza, chiavi delle memorie dei piani e verifica della loro validità. Un fastpath valido può riusare un piano senza richiamare il pianificatore; un nuovo piano, invece, è ancora generato da un LLM e può variare anche con temperatura bassa o seme impostato. Lo stesso vale per la prosa della risposta. Le prove devono quindi distinguere l'identità del piano riusato dalla qualità di una nuova generazione.

Cosa fa meglio un agente da terminale

Precisazione necessaria: come compagno in tempo reale di chi scrive codice dentro un repository, un agente da terminale maturo resta superiore, con un ecosistema di estensioni che Metnos non ha né insegue. Ma Metnos non è definito dalle attività domestiche né da un solo verticale. È un'architettura operativa che tiene pianificazione, capacità eseguibili, autorità, memoria e tracciamento dentro un unico confine governato. Posta, foto e calendario sono usi di riferimento; mantenere Metnos attraverso i suoi flussi GitHub è già un uso non domestico. I due tipi di sistema convivono benissimo.

3 · Nove casi, un solo sistema

I casi che seguono non sono moduli separati e non definiscono un perimetro esaustivo del prodotto: usano lo stesso pianificatore, gli stessi contratti e le stesse guardie. La linea li tiene insieme; ogni tappa mostra soltanto la richiesta, la catena rilevante e il limite operativo che cambia. Il caso di manutenzione GitHub mostra deliberatamente che l'architettura non è confinata alla casa.

01

«Riassumi le mail di oggi»

“ Dammi il riassunto delle mail importanti di oggi.

CATENA DI EXECUTOR

read_messages → classify_entries → filter_entries → describe_entries → risposta

METNOS

3 mail oggi: l'avvocato chiede il documento entro venerdì; la banca conferma il bonifico di ieri; la sorella sul pranzo di domenica. Vuoi che provi a rispondere a una di queste?

capability mail:read · **conferme** nessuna — i verbi di sola lettura saltano il giudizio · **latenza** ~3 s

02

«Rispondi tu all'avvocato»

“ Ok, rispondi tu all'avvocato: digli che invio il documento entro giovedì.

Una mail in uscita verso un destinatario esterno non parte mai in silenzio, a nessun livello di autonomia. Metnos compone la bozza e su Telegram arriva questa bolla con due bottoni:

Posso inviare una mail?

a avvocato@studio.example · oggetto: «consegna documento» · ~140 caratteri

reversibile: no · classe: mail:send

✓ Approva

✗ Rifiuta

Toccate Approva: il daemon verifica che il token sia ancora pendente, non scaduto, e che *chi decide sia lo stesso mittente accoppiato che ha chiesto*. Poi: «Spedita alle 11:34. Registrazione dell'azione #1247.»

capability mail:send (irreversibile) · **token** monouso, TTL 600 s · **identità** chi approva == chi ha chiesto

Questo è un esempio di proposta. Il testo effettivo dipende dall'azione e dal canale, mentre etichette e pulsanti sono risolti nella lingua dell'utente dal catalogo i18n. La decisione resta legata alla proposta e all'utente che l'ha richiesta. Vedi `approval_ux`.

03

«Ordina queste foto per anno e luogo» — e poi «annulla»

“ *Ordina tutti i file immagine in ~/images creando sottocartelle per anno, premettendo al nome data e luogo di scatto. Se il luogo non c'è, metti «unknown».*

Non esiste uno strumento «ordina foto»: il pianificatore ne compone quattro della libreria, collegando l'uscita di ognuno all'ingresso del successivo. I dati non transitano dal modello: passano per riferimento (*from_step*).

```
1 find_files(base_path=~"/images", patterns=["*.jpg", "*.png", "*.heic"],
recursive=true)
```

→ 98 entries (con size e mtime già inclusi)

```
2 filter_entries(from_step=1, kind="image")
```

→ 98 entries · filtro in memoria, conferma il tipo dichiarato dal MIME

```
3 get_files(from_step=2, fields=["dates", "place"])
```

→ 98 entries arricchite: EXIF → data; GPS → luogo via reverse-geocode (cache locale); senza GPS → "unknown"

```
4 move_files(from_step=3, dst_template=~"/images/{anno}/{data}_{luogo}_{nome}")
```

→ 98 spostati, 0 errori · ogni coppia sorgente→destinazione registrata per l'annullamento

Quattro executor indipendenti, scelti e collegati dal pianificatore. Lo stesso flusso gestisce una o molte foto.

Il giorno dopo, cambio di idea:

“ *Annulla l'ultima operazione.*

METNOS

Ho annullato l'ultima operazione: ho riportato 98 file ai loro percorsi originali.

annullamento a più stadi: prima i file, poi le cartelle create — solo se rimaste vuote · **tempo di annullamento** ~3 s su 98 file · **idempotente** un secondo «annulla» non fa nulla

ONESTÀ

Non tutto è annullabile, e Metnos non finge il contrario: una mail spedita non si richiama. L'annullamento dichiara «X operazioni non annullabili» e inverte il resto. Ogni spostamento segue COPIA → verifica → CANCELLAZIONE: mai una cancellazione senza copia confermata.

04

«Ogni mattina, il registro scolastico — alla figlia, non a me»

Il portale della scuola richiede login. Il riassunto deve arrivare sul Telegram della figlia, accoppiata come ospite. Tre messaggi di preparazione, poi va da solo.

“ *Aggiungi credenziali per portale-scuola.example: utente roberto.b, password*

● ● ● ● ● ● ● ●

Metnos riconosce un valore che somiglia a una password e apre una carta dedicata: propone di cifrare il dato in una cassaforte locale e di *cancellare il valore in chiaro dal log del turno* (<REDACTED:cred>). Approvato, prova un login di test.

“ *Ogni mattina alle 7 cerca le novità sul portale (compiti, voti, circolari), leggi anche i PDF allegati, e manda il riassunto delle 5 più rilevanti alla figlia su Telegram.*

CATENA DI EXECUTOR

```
pianificazione 07:00 · open_sites → login_sites (executor agentificato) → act_sites →  
read_urls_pdf → group_entries → describe_entries → send_messages(to_user="figlia")
```

METNOS

Scuola — riepilogo del giorno

1. Sciopero docenti (circolare 198) — lezioni sospese al mattino
2. Verifica di matematica spostata alla settimana successiva
3. Compiti del fine settimana: capitoli 4–6 + saggio breve
4. Riunione coordinatori venerdì 9/05, ore 15:30
5. Pubblicato il voto di latino

credenziali cifrate in locale, mai nel codice né nei log · **intelligenza interna** login_sites adatta il percorso, ma non può cambiare mandato · **cookie** riusato ~14 giorni; alla scadenza nuovo accesso automatico · **LLM** 1 sola chiamata per esecuzione (il riassunto) — l'HTML si estrae in modo deterministico · **latenza** ~25–40 s a esecuzione

ONESTÀ

Se anche il nuovo accesso fallisce (password cambiata sul portale), l'attività viene messa in pausa e l'utente principale riceve un avviso — non un silenzio, non un resoconto inventato.

05

Da una cartella eterogenea a un dossier verificabile

Una sola richiesta attraversa PDF, Word, Excel e CSV sul PC associato; unisce ricerca temporale, deduplicazione, estrazione, controllo incrociato e tre risultati:

“ Nella cartella Documenti/Progetto Atlas, trova PDF, documenti e fogli di calcolo modificati negli ultimi 60 giorni. Elimina logicamente i duplicati confrontando firme e contenuto, estrai scadenze, importi, persone e decisioni, quindi crea un riepilogo ordinato per scadenza e un foglio con origine, data, responsabile, importo e livello di confidenza. Segnala dati contraddittori e file illeggibili. Salva i risultati in una nuova sottocartella, genera anche un archivio compresso, ma non sovrascrivere né cancellare nulla senza approvazione.

CATENA DI EXECUTOR

```
find_files → filter_entries (60 giorni) → read_files (auto + firme) → extract_entries →  
sort_entries → describe_entries → create_dirs + write_files → create_files_spreadsheet  
→ find_files → compress_files
```

METNOS

6 file candidati · 1 duplicato eliminato logicamente · 5 sorgenti uniche · 7 righe estratte. Contraddizione segnalata: budget approvato €120.000 con scadenza 30/09, revisione €135.000 con scadenza 15/09. Un PDF è illeggibile. Creati in una nuova sottocartella: riepilogo.md, dati_estratti.xlsx e risultati.zip.

formati PDF + DOCX + XLSX + CSV, letti in un unico flusso · **controlli** hash binario + firma + hash del contenuto, anomalie deterministiche · **sicurezza** produce soltanto file nuovi; nessuna sovrascrittura o cancellazione · **collocazione** i file restano sul PC Windows associato

ONESTÀ

Prova E2E sul PC Windows reale: 11 passaggi su 11 riusciti in 74,9 s; foglio con intestazione più 7 righe, archivio ZIP con i due risultati richiesti e impronta degli 8 file in ingresso invariati. Il livello di confidenza non trasforma una stima in certezza: per questo file illeggibili, duplicati e valori in conflitto restano visibili nel riepilogo. La ricevuta finale elenca cartella e tre artefatti; non espone la tabella tecnica dell'archiviatore.

06

«Posso leggere e scrivere file Excel?» — il sistema spiega se stesso

“ È possibile leggere e scrivere file Excel? Quali limiti ci sono?

Questa non è un'operazione: è una domanda su Metnos. Il Tutor la separa dal pianificatore, confronta la domanda con il catalogo firmato e con la documentazione locale ammessa, quindi compone una risposta nella lingua corrente. Non esegue executor e non apre file.

CATENA DI EXECUTOR

domanda → embedding locale → manifesti ammessi + documentazione → filtro del ruolo → risposta fondata

METNOS

Sì. Il catalogo installato distingue la lettura di cartelle di lavoro .xlsx dalla creazione o scrittura di fogli. Puoi indicare il foglio da leggere; per il risultato puoi fornire colonne e righe provenienti da un passo precedente. Questa risposta descrive le capacità disponibili: non ho aperto o modificato file.

fonti descrizioni degli executor realmente ammessi · **lingue** fonti IT/EN, retrieval cross-lingua e risposta nella lingua della chat · **autorità** nessuno strumento e nessuna capacità di esecuzione

ONESTÀ

«Leggi tutti i miei fogli Excel» non è una domanda Tutor: passa al normale motore operativo con i suoi controlli. Il confine evita che una spiegazione diventi un'azione per equivoco. Il capitolo seguente mostra il disegno completo.

07

«Trova le foto in montagna» — semantica visiva, in casa

Decine di migliaia di foto su un disco di rete. La prima volta Metnos propone di indicizzarle: l'indicizzazione gira in background e avvisa su Telegram a lavoro finito. Da quel momento:

“ Trova le foto in montagna.

CATENA DI EXECUTOR

find_images_indices(query="montagna") → 20 anteprime + galleria web (100)

“ Le foto della figlia al mare?

CATENA DI EXECUTOR

find_persons_indices (volti: dopo un'etichettatura una-tantum «chi è chi») → galleria

modelli embedding di scena + riconoscimento volti + EXIF — tutto nel processo, sul vostro server · **privacy** nessuna foto lascia casa; l'indice è un file SQLite ispezionabile · **interfaccia** anteprime in chat, galleria completa nel browser

ONESTÀ

Una ricerca che combina persona e scena deve indicare quali segnali ha applicato. Se la fusione non è disponibile, Metnos restituisce un esito parziale esplicito invece di presentarlo come ricerca completa.

08

Manutenzione GitHub senza demone: due attività programmate

Il supporto del progetto stesso è gestito così — niente servizio dedicato, niente codice ad hoc: due richieste in linguaggio naturale, registrate come attività ricorrenti, che il pianificatore trasforma nelle solite catene di executor.

“ Ogni 30 minuti: trova le segnalazioni nuove del repository, scarta quelle già nel database locale, cerca per ciascuna le issue simili già risolte, classificala e analizzala col tier *frontier*, salva la bozza di risposta con stato «prepared», e avvisami.

“ Dopo la mia approvazione: leggi dal database le issue «approved» non ancora pubblicate, pubblica la risposta come commento su GitHub, segnala «posted».

CATENA DI EXECUTOR

```
find_issues_github → find_issues (dedup nel db locale) → classify_entries +
consult_frontier → write_issues("prepared") → approvazione umana → send_messages_github
→ write_issues("posted")
```

macchina a stati new → prepared → approved → posted: il doppio invio lo impedisce lo stato, non la speranza · **consenso** niente raggiunge GitHub senza un «approva» esplicito · **programmazione** una frase: «crea un'attività ricorrente ogni 30 minuti: ...»

ONESTÀ

È un esperimento dichiarato: un'istanza di Metnos che aiuta a mantenere Metnos. Se il tier *frontier* non è disponibile o supera il budget, l'issue resta «new» e viene segnalata per il trattamento manuale — mai una risposta inventata, mai un'omissione silenziosa.

09

Dalla chat a un tuo PC, con la destinazione che resta

Gli stessi comandi in linguaggio naturale, ma l'esecuzione può spostarsi su un PC che avete *appaaiato*. La chat parte con destinazione **server**; a ogni cambio compare una pillola **«destinazione ora: ...»** che dice dove finisce il prossimo comando se non nominate nessun PC (destinazione appiccicosa).

Metnos · chat di Roberto

Quante righe di codice ci sono nel progetto?

server

Il conteggio è stato eseguito sul server.

```
primo turno, nessun PC nominato → destinazione = server
```

Elenca la cartella Documenti sul mio **portatile-ufficio**

☐ riconosciuto **PORTATILE-UFFICIO** → instrado, e da ora è la destinazione

☐ **PORTATILE-UFFICIO**

Su **PORTATILE-UFFICIO**, in **Documenti**:

fatture/	(cartella)
contratti/	(cartella)
appunti.txt	4 KB
bilancio.xlsx	88 KB

☐ **destinazione ora: PORTATILE-UFFICIO**

Comprimila in zip

↳ nessun PC nominato → resta l'ultima destinazione: **PORTATILE-UFFICIO**

☐ **PORTATILE-UFFICIO**

Fatto sul portatile: creato **Documenti.zip** (1,2 MB) accanto alla cartella.

Quanto spazio libero c'è sul **fisso-casa**?

☐ **FISSO-CASA · non connesso**

FISSO-CASA non è connesso (nessun battito da oltre un minuto). Non eseguo altrove di nascosto: accendilo e riprova, oppure dimmi «sul server».

```
controllo di connessione prima di ogni invio → errore esplicito, mai un ripiego silenzioso
```

destinazione appiccicosa: resta sull'ultimo PC finché non ne nominate un altro o non dite «sul server» · **connessione** verificata prima di ogni invio (battito) — se il PC è spento, errore esplicito, nessun ripiego di nascosto · **appaiamento** solo i PC che avete registrato voi

ONESTÀ

Metnos non esegue mai «alla cieca» su un PC che non risponde: niente battito recente, niente esecuzione. Vi dice che il dispositivo è scollegato e vi lascia scegliere — accenderlo, ripetere altrove, o restare sul server.

4 · Il Tutor: Metnos sa spiegare Metnos

Un sistema governato serve davvero solo se l'utente può chiedergli che cosa sa fare, perché si è fermato e come si usa una funzione. Il **Tutor** rende leggibili capacità e procedure senza trasformare la guida in un altro agente onnipotente.

Conoscenza compilata, non una risposta a memoria

Il Tutor indicizza le descrizioni multilingue dei manifesti realmente ammessi, tutte le pagine HTML pubbliche e indicizzabili distribuite con Metnos, alcune procedure curate e proiezioni esplicite dei registri di sistema. La pubblicazione e il Tutor condividono lo stesso inventario validato: aggiungere o aggiornare una pagina pubblica aggiorna la fonte al ciclo di compilazione successivo. Testi, vettori e metadati vivono in un catalogo SQLite firmato e sostituito atomicamente. La `directory internal/` non entra nel corpus in alcun ambiente: le informazioni utili devono prima essere verificate e trasferite nella documentazione pubblica. Il Tutor non scandisce Documenti, posta, calendari, registri, note tecniche o conversazioni dell'utente.

Se una domanda nomina esattamente un documento dell'inventario pubblico — per nome, percorso relativo o indirizzo canonico — il Tutor riconosce quella fonte prima del recupero semantico e limita la risposta alle sue sezioni. La parola «file» non basta: un nome sconosciuto resta nel normale dominio dei file, mentre una richiesta di cancellazione resta un'azione anche quando il documento è riconosciuto.

CATENA DI EXECUTOR

fonti locali ammesse → unità tipizzate → embedder locale configurato → catalogo firmato → risposta senza strumenti

Domande che risolve oggi

“ *Come leggo tutte le caselle email senza indicarle una per una?* ”

Recupera la descrizione corrente di `read_messages` e spiega il valore `account="all"`.

“ *Appaiamento dei dispositivi: vorrei capirne la logica.* ”

Usa la guida amministrativa solo se l'identità autenticata è autorizzata; il testo riservato non raggiunge il modello per gli altri utenti.

“ *`read_messages?`* ”

Anche una forma telegrafica può essere riconosciuta semanticamente, senza inserire quella frase in una tabella.

“ *Che cosa contiene il file `Metnos_Prospettive_Estese_v1.html`?* ”

Riconosce il nome nell'inventario pubblicato, riassume quel documento e ne offre il collegamento canonico.

Tre cose che non fa

Non esegue da solo. «Leggi tutte le mail» prosegue nel motore normale. In una richiesta mista, il Tutor spiega e chiede conferma prima di affidare al motore la clausola d'azione scritta dall'utente.

Non completa i vuoti. Se le fonti non bastano, dichiara la lacuna.

Non confonde i ruoli. Audience e identità arrivano dal canale autenticato, mai dalle parole della domanda.

Stato corrente, con osservazioni delimitate

Le fonti firmate che descrivono Servizi, Executor, Dispositivi e attività programmate possono richiamare una sonda di sola lettura registrata dal runtime. Così il Tutor distingue la descrizione di una pagina dal suo stato corrente. Ogni osservazione è limitata per tempo e dimensione, applica l'identità dell'utente e dichiara se il dato è completo, parziale, scaduto o indisponibile. Il modello non sceglie la sonda e non riceve segreti, chiavi, endpoint o risultati privati.

Un riscontro personale, non una regola per tutti

Un riscontro positivo può associare semanticamente la domanda alla fonte che ha prodotto la risposta, soltanto per lo stesso utente. Un riscontro negativo rimuove l'associazione e registra una lacuna tipizzata. Il testo della domanda non viene conservato in chiaro; associazioni e prove scadono, sono invalidate se cambia la fonte o l'embedder e restano fuori dal pianificatore e dalle cache dei piani.

Una lingua nuova non richiede di duplicare la guida

Il catalogo conserva la lingua di ogni fonte e il modello di rappresentazione semantica è multilingue. Quando una nuova lingua viene ammessa e selezionata, il recupero cerca prima le fonti in quella lingua, poi quelle inglesi e infine le altre traduzioni disponibili. Il compositore riceve sempre la lingua della risposta come dato esplicito: il testo dell'istruzione usata come

ripiego non decide la lingua dell'utente. Per offrire una lingua come opzione completa restano comunque necessari il relativo pacchetto i18n e la revisione delle traduzioni.

5 · Dove vive e come gli si parla

Metnos gira su una macchina Linux vostra. **I livelli LLM sono ruoli astratti, non modelli fissati: un endpoint su CPU, un modello locale o remoto che già serve e un eventuale ripiego di frontiera sono percorsi pienamente supportati. Telefoni e portatili sono client: la mente, la memoria e il registro delle azioni vivono su una macchina sola.**

Due superfici, una mente

Il browser: una chat servita dal server stesso (porta 8770), con le anteprime delle foto, la galleria, e i cruscotti di amministrazione (proposte del synt, executor, esecuzioni, turni). **Telegram:** lo stesso cervello in tasca, con le carte di conferma come bottoni in linea. Per chi automatizza, l'API HTTP (/agent/turn, anche in streaming) è la stessa porta che usa la chat.

La chat web mostra anche i badge di riscontro ✓/✗ per ogni risposta. Nei turni del Tutor il verdetto può affinare il recupero per quell'utente oppure registrare una lacuna, senza cambiare il comportamento degli altri account.

La conversazione può passare da un dispositivo all'altro. Se lo stesso utente apre la chat mentre una sessione web è già attiva altrove, Metnos mostra tre scelte: *Continua la sessione precedente* trasferisce sul dispositivo corrente quella conversazione e il suo storico; *Rendi attiva questa sessione* mantiene la conversazione aperta localmente e revoca l'altra sessione; *Annulla* non cambia nulla. Le conversazioni non vengono fuse e il dispositivo revocato perde il diritto di scrittura. Sessioni, storico e scelta attiva restano separati per utente.

Chi siete, per il bot: l'appaiamento

Telegram è una superficie pubblica: chiunque conosca il nome del bot può scrivergli. Metnos lega ogni coppia (*canale, mittente*) a un livello di autonomia con un codice firmato monouso: /pair PAIR.<codice> (Ed25519, scade in pochi minuti) per gli appaiamenti decisi dal proprietario; /start <token> per gli ospiti creati dall'interfaccia di amministrazione. Niente account centrale, niente password: una rubrica per canale e per mittente che solo il proprietario modifica. Vedi appaiamento.

La lingua è un dato, non una costante

Ogni messaggio visibile, ogni descrizione di strumento e ogni istruzione al modello vive come *dato per-lingua*: italiano e inglese sono validati, una lingua nuova si aggiunge per traduzione dei pacchetti, senza toccare codice (prompts_cli add-language <codice> prepara la struttura; la traduzione si rivede e si promuove a mano). Per chiarezza: oltre IT e EN, oggi, è terreno non testato. Vedi multilang.

6 · Cosa succede quando dice «posso?»

È il pezzo di esperienza più importante del progetto. Prima di eseguire un passo, Metnos applica i controlli di sicurezza e le regole della capacità. Quando le regole richiede una decisione umana, il passo resta sospeso finché l'utente che lo ha chiesto non approva o rifiuta.

FASE	COSA FA
1 · Pianificatore	Prepara il passo (executor e argomenti) in memoria. Niente è ancora eseguito.
2 · Vaglio	Prima la <i>guardia binaria</i> : percorsi vietati (~/.ssh, /etc...) e pattern di shell quasi irrecuperabili (rm -rf /, mkfs, fork bomb) sono negati a prescindere, a ogni livello di autonomia. Poi il <i>giudice graduato</i> : un punteggio di allineamento ai vostri telos (i fini scritti in TELOS.md).
3 · Regole	Classe della capacità, livello di autonomia e concessioni persistenti producono uno fra allow_silent, approval_required e deny.
4 · Dialogo	Il canale mostra la proposta e le scelte localizzate disponibili. La decisione è legata al turno e all'utente richiedente; una proposta scaduta o già consumata non esegue nulla. Solo dopo un'approvazione valida parte l'executor, con la regola di esecuzione prevista dal suo contratto.

Controllo senza ambiguità. Approvazione, rifiuto, scadenza e consumo della proposta sono stati distinti. Eventuali concessioni persistenti devono essere create e revoked esplicitamente dalle regole: la semplice approvazione di un passo non viene documentata come autorizzazione generale. Vedi approval_ux.

E quando ha agito, può tornare indietro. L'annullamento non è un LLM che «ci prova»: è un catalogo chiuso di operazioni inverse dichiarate nel manifesto di ogni executor mutante (scambia sorgente/destinazione, cancella i creati, ripristina il backup del contenuto, cancellazione per identificatore). Conteggi verificabili: se indica tre operazioni annullate, erano tre.

Limiti che l'autonomia non rimuove

La guardia binaria non è configurabile da file: percorsi vietati e schemi catastrofici cambiano soltanto con una modifica del codice. Anche la generazione ha un confine preciso: Synt produce candidati executor, non riscrive se stesso, il Vaglio o il runtime. Ammissione, prove e ciclo di vita decidono poi se e quando un candidato può diventare operativo; le operazioni protette conservano i loro consensi.

7 · La sua memoria: cosa ricorda e cosa no

Quattro depositi distinti, con durate e regole di scrittura diverse — tabelle *SQLite* e file di testo che potete aprire, non un vettore opaco da qualche parte.

DEPOSITO	ORIZZONTE	COSA CONTIENE	REGOLA
Scratchpad	il turno	Le osservazioni dei passi intermedi.	si svuota a fine turno
Storico turni	giorni	«Cosa ti ho chiesto martedì scorso?»	un file per turno, alimenta i cruscotti
Preferenze esplicite	finché rimosse	Lingua, tono, lunghezza delle risposte, unità e opzioni di navigazione.	vocabolario chiuso, valori scelti e separati per utente
Mnestoma	su di sé	Quali strumenti hanno collaborato, con che esito; e i tentativi rimasti a metà.	il substrato che il synt legge per proporre

Lo stato del turno, lo storico operativo, le preferenze della persona e la memoria del funzionamento del sistema hanno scopi e regole distinti. Non vengono riuniti in un profilo generico.

Metnos oggi non deduce né salva automaticamente fatti liberi sulla persona. Le sole preferenze persistenti dell'utente appartengono a un vocabolario chiuso e sono impostate esplicitamente; ogni utente conserva i propri valori. Lo storico dei turni resta una registrazione operativa, non viene promosso a profilo personale. Un tentativo che non trova strumenti adeguati può invece lasciare un *proto-mnest*: una lacuna del sistema, non una memoria biografica. Vedi mnestoma e scratchpad.

8 · Come cresce: Synt, skill e servizi di esecuzione

Un *executor* è un piccolo programma Python che fa una cosa sola, con un manifesto che ne dichiara argomenti, capacità e profilo di isolamento. Una volta firmato è un artefatto stabile: non impara, non cambia. L'intelligenza di crescita sta altrove — nel **sintetizzatore**.

La sintesi in cinque stadi

STADIO	COSA PRODUCE
1 · Nome	dal vocabolario chiuso — livello middle
2 · Firma	schema degli argomenti, capacità e procedura inversa
3 · Test	4–6 prove di nascita: felice, vuoto, invalido
4 · Descrizione	le istruzioni dello strumento e le parole di affinità
5 · Codice	livello wise, locale — <code>def invoke(args) → liste</code>

All'uscita, il **filtro di ammissione** in sette livelli controlla firma, sovrapposizione di affinità, maturazione, isolamento, prova operativa, coerenza fra descrizione e codice e registrazione a sola aggiunta; seguono il ciclo di valutazione e la firma Ed25519 previste per il candidato.

Cinque istruzioni piccole e focalizzate, invece di una monolitica, separano le responsabilità e producono artefatti che si possono ispezionare e provare separatamente. Solo lo stadio 1 vede il vocabolario chiuso; solo il 5 scrive codice.

Quando un turno incontra una lacuna, la sintesi reattiva prova soltanto a **comporre executor già firmati**; non genera codice in silenzio. Le richieste di un nuovo executor e le altre proposte di cambiamento entrano nel ciclo governato di `change_intent`, dove restano visibili e richiedono le verifiche e le decisioni previste prima di modificare il catalogo. La manutenzione introvertiva notturna attiva segnala soltanto relazioni mnest obsolete o orfane da deduplicare; non genera generalizzazioni o specializzazioni. Vedi synt e ciclo di vita delle modifiche.

Skill e servizi di esecuzione: due assi che non si confondono

Una **skill** è un gruppo governato di capacità che può essere attivo, disattivato o dormiente. Il registro distingue nucleo, skill mantenute da Metnos e skill importate. Dove esiste un controllo deterministico dei prerequisiti, una skill senza credenziali, indice o archivio necessario resta visibile ma i suoi executor non vengono proposti. Le skill si gestiscono da riga di comando o chiedendolo in chat («quali skill ho?», «disattiva il web dal prossimo turno»).

Il **servizio di esecuzione**, chiamato `backend` nel codice, è l'altro asse: stabilisce *come* un'azione raggiunge un servizio concreto. File, cartelle, eventi e contatti usano un risolutore centrale; la posta usa il risolutore degli account; GitHub conserva executor qualificati dal fornitore. In ogni caso la scelta deriva da configurazione e contesto esplicito, non da una preferenza libera del modello. Vedi skill e servizi di esecuzione.

Amministrazione del sistema operativo

La skill `system` comprende interprete dei comandi, `sudo`, pacchetti e montaggi di rete. Ogni azione privilegiata richiede il giudizio del Vaglio e le autorizzazioni previste. La skill può essere disattivata: finché resta spenta, Metnos non usa queste capacità sul sistema operativo.

9 · Persone e dispositivi: utente principale, ospiti, executor remoti

Metnos non è multi-tenant in senso SaaS: è una casa sola. Dentro quella casa c'è un **utente principale**, indicato nel codice come `host` — il proprietario del server e custode della chiave di firma — e zero o più **ospiti**, ognuno col suo livello.

LIVELLO	PER CHI	COSA PUÒ
ReadOnly	Un ospite, un canale delicato	Solo executor di lettura; ogni scrittura viene rifiutata con garbo prima ancora di arrivare al pianificatore.
Supervised	L'uso quotidiano	Tutto il turno; le azioni delicate alzano la carta di conferma sul canale stesso.
Piena (Full)	L'utente principale	Il perimetro più ampio ammesso dalle regole. I percorsi vietati restano vietati anche qui.

La scena 4 mostra il modello al lavoro: l'utente principale configura, l'ospite riceve, le righe del registro restano separate per associazione. Ogni canale ha la sua porta, la sua chiave, la sua storia.

Executor remoti: dove può arrivare il codice

Un solo server, ma non un solo filesystem: tante cose su cui vorreste agire (i file del portatile, un'applicazione aperta, uno schermo) vivono altrove. Server, regole e memoria restano sul server; *alcuni executor* girano su dispositivi registrati — un client sottile con identità Ed25519 propria, ammesso con un codice firmato monouso, revocabile dal server in un gesto.

ONESTÀ

Il client remoto è un piccolo programma Rust con isolamento e compilazione firmata; sul server il canale di invocazione applica idempotenza su rete intermittente, protezione dai replay e consegna differita del risultato dopo una disconnessione. I test E2E coprono il confine server-dispositivo. Dettaglio: executor remoti.

10 · Cosa abilita il catalogo, cosa vieta il core

Il primo elenco descrive il set di *executor* dell'istanza di riferimento, non un dominio intrinseco di Metnos. Il secondo descrive confini architettureali che restano validi quando il catalogo cambia.

✓ Cosa abilita oggi il catalogo di riferimento

- Turni multi-passo da browser e Telegram, con piani fino a 12 passi
- Posta: legge, riassume, archivia e risponde; ogni invio richiede una scheda di conferma
- File: cerca, filtra, sposta, comprime ed estrae, con annullamento dichiarato
- Foto: indicizza in casa (scena, volti, EXIF) e cerca per significato
- Google Workspace con un solo OAuth: Gmail, Calendar, Drive, Docs, Sheets
- GitHub: issue, pull request e file del repository come entità pienamente supportate
- Web: ricerca e lettura tramite servizi ospitati localmente, compresa la georeferenziazione
- Compiti ricorrenti in linguaggio naturale («ogni mattina alle 7...»)
- Più utenti sullo stesso bot, con autonomie e registrazioni separate
- Executor remoti su dispositivi Linux/Windows registrati, per executor esplicitamente abilitati alla piattaforma
- Può generare candidati executor mediante una richiesta governata; la manutenzione notturna corrente deduplica le relazioni obsolete
- Parla italiano e inglese per costruzione; ogni stringa è un dato
- Registro a sola aggiunta delle azioni e pannelli di osservabilità

— Cosa vieta il nucleo, o il catalogo corrente non offre ancora

- Agire fuori casa senza consenso: le capacità in uscita passano dalla carta
- Spendere sul tier di frontiera oltre il tetto configurato
- Toccare `~/ .ssh, /etc, ~/ .gnupg` e simili: lista cablata nel codice, non in un file di configurazione
- Eseguire comandi catastrofici (`rm -rf /`, `mkfs`, `fork bomb`), a qualunque autonomia
- Modificare se stesso: il synt genera solo executor, mai il runtime o il vaglio
- Installare un executor senza la vostra conferma esplicita
- Ascoltare da un microfono (canale voce in attesa, per scelta)
- Usare un sito oltre il mandato concesso alle sue credenziali
- Eseguire su dispositivi remoti executor non firmati, non registrati o non dichiarati compatibili col sistema operativo
- Presentare come completa una ricerca foto quando uno dei segnali richiesti non è disponibile
- Garantire lingue oltre italiano e inglese: sono aggiungibili come pacchetti, ma non ancora collaudate

Regola trasversale: **nessun fallimento silenzioso**. Cap raggiunto → lo dice (quanti usati, quanti disponibili) e chiede se allargare. Esito parziale → contato come parziale. Mai un «fatto» che non corrisponde alla realtà.

11 · Stato operativo

Metnos è un sistema pre-1.0 installabile e operativo, non un prodotto rifinito. Contratti, sicurezza e verifiche governano gli aggiornamenti. Il catalogo generato, il codice e le prove automatiche sono le fonti dello stato corrente.

AREA	STATO OPERATIVO
Capacità	Posta, file, foto, web, Workspace, attività e dispositivi remoti sono esposti come executor con contratti dichiarati. Il riferimento dei domini spiega che cosa chiedere con esempi naturali; il catalogo generato resta la fonte tecnica per nomi e conteggio.
Programma di installazione	Incluso. Il percorso <i>managed</i> prepara servizi, dati i18n ed executor firmati, profila l'hardware e verifica l'avvio e l'endpoint di salute. La verifica operativa si completa dopo il primo accesso con una richiesta innocua nella chat. Il percorso <i>custom</i> collega endpoint già gestiti dall'operatore.
Repository pubblico	Esportazione deterministica dei componenti necessari all'installazione e all'esercizio, sottoposta a controlli prima della pubblicazione.
Maturità	Pre-1.0: interfacce esterne e dati persistenti richiedono cautela negli aggiornamenti. Fallimenti parziali, dipendenze non disponibili e capacità non supportate devono essere dichiarati, mai nascosti.

Provarlo

Il requisito vero è l'hardware: una macchina che regga un LLM locale capace. I livelli sono ruoli astratti — un endpoint su CPU o un modello già in servizio vanno bene; un modello più debole significa pianificazione più debole, non un'installazione rotta. Nessuna GPU è richiesta per principio.

```
$ git clone https://github.com/brunialti/metnos.git && cd metnos
$ bash install/bootstrap.sh --check      # controlla i requisiti; può inizializzare .venv
$ bash install/bootstrap.sh              # interattivo, sei fasi, idempotente
```

La descrizione da mezzo euro

Se portate via una cosa sola: Metnos è un'architettura ospitata in proprio per un agente governato, con cinque impegni peculiari — chiede prima delle azioni che richiedono conferma, costruisce strumenti dentro regole verificabili, mantiene deterministici instradamento, regole e tracciamento senza fingere che la prosa di un LLM sia riproducibile, concentra l'intelligenza adattiva dentro executor a mandato ristretto e porta executor firmati sui dispositivi senza spostare decisione e registrazione dal server. Se questa combinazione vi incuriosisce, siete il pubblico giusto.

12 · Glossario minimo & letture successive

Executor

Uno strumento del catalogo: file Python, manifesto, firma Ed25519 e profilo di isolamento. Vettoriale per costruzione: liste in ingresso, liste in uscita.

Executor agentificato

Un executor con un ciclo interno limitato di osservazione e azione. Può usare un LLM per disambiguare il percorso, ma mandato, autorità, ingresso, uscita e verifica restano quelli del contratto pubblico.

Executor remoto

Un executor eseguito da un client su un dispositivo registrato. Il server pianifica, autorizza, firma e registra; il dispositivo verifica ed esegue.

Synt

Il sintetizzatore: durante il turno prova a comporre executor firmati; una richiesta governata può generare un candidato nuovo. La manutenzione introvertiva attiva è limitata alla deduplicazione; generalizzazione e specializzazione automatiche sono ritirate.

Vaglio

Il valutatore in due fasi fra un'azione proposta e la sua esecuzione: guardia binaria non negoziabile + giudice graduato sui telos.

Telos

I vostri fini, scritti in `TELOS.md` come tendenze morbide. Il giudice misura le azioni proposte contro di essi.

fastpath

Il sentiero veloce appreso: una richiesta già risolta viene rigiocata senza chiamate LLM. Stesso esito, latenza e costo minimi. Quando una forma si ripete per un intero gruppo di richieste simili, il piano generalizzato diventa un *autopath*.

Mnestoma

La memoria che il sistema tiene su di sé: co-attivazioni riuscite (mnest) e tentativi rimasti a metà (proto-mnest). Il substrato delle proposte di crescita.

Skill / servizio di esecuzione

Due assi ortogonali: la skill governa *se* una capacità è attiva; il servizio, chiamato `backend` nel codice, governa *come* raggiunge un fornitore concreto. La scelta appartiene alla configurazione e al contesto esplicito, non a una preferenza libera del modello.

Appaiamento

L'associazione autenticata di un canale o dispositivo a un utente Metnos. Ruoli, preferenze, sessioni e cronologie restano indipendenti per utente.

Carta di conferma

Il dialogo localizzato che spiega l'azione proposta, il bersaglio e le conseguenze, quindi presenta soltanto le scelte ammesse dal flusso corrente.

Continua a leggere

Riferimento dei domini

metnos.com/it/domains

Tutti i domini operativi, i loro confini e frasi di esempio pronte da usare in chat.

RIFERIMENTO · 15 MIN

Guida all'architettura

metnos.com/it/architecture/

La visione d'insieme e l'atlante completo dei componenti: topologia, flusso, autorità e contratti.

GUIDA COMPLETA · 20 MIN

L'interfaccia

metnos.com/it/interface

La chat web, Telegram, le sessioni per utente e la mappa delle pagine di Settings.

GUIDA · 8 MIN

Una nota sul codice

metnos.com/it/code.html

Che cosa pubblica il repository e come si confronta con un'istanza installata.

NOTA · 5 MIN

Metnos — *Visita guidata e kit essenziale*

mētīs + noûs · pensato per essere letto in venti minuti · l'Architettura è il
seguito naturale

github.com/brunialti/metnos · metnos.com · metnos@metnos.com